

OptSem-C: Benchmarking Projection Precision in Public Query-Optimizer Contracts

Haoyi Zhang

Xi'an Jiaotong-Liverpool University
Suzhou, China
hyeliozhang@gmail.com

Huaijin Ran

Nanyang Technological University
Singapore, Singapore
huaijin003@e.ntu.edu.sg

ABSTRACT

Comparisons of analytical SQL optimizers can lose contract distinctions before any runtime metric is meaningful: the denominator is often a coarse public vocabulary rather than a declared public-contract relation. Engine-selection, migration, and performance-debugging studies compare labels such as pushdown, join, or cost based. Such labels can merge local rewrites, connector delegation, service stages, and runtime decisions into one apparent capability; a study can then compare different query-processing phenomena while believing the systems have been normalized.

This paper presents OptSem-C, an executable benchmark for optimizer-contract comparison with projection counter-witnesses and finite repair certificates. A contract rule is a guarded statement over query features with explicit public modality, operator, action variant, optimizer layer, execution placement, decision time, observability, version, and public evidence. The semantics separates behavioral worlds from evidential support, defines a conflict-aware state join, and treats lossy comparison as a projection kernel over reference public-contract signatures. OptSem-C lets a study publish the vocabulary it preserves, the equivalences it assumes, and the repair fields needed to make those assumptions safe.

On a source-linked corpus spanning seven analytical SQL engines, 287 admitted rules, and 4,216 executable probes, OptSem-C shows that denominator-level precision loss is common and repairable. Keyword and operator-only vocabularies declare thousands of engine-probe equivalences; 254 and 238 of those equivalences, respectively, are separated by source-supported reference signatures. These witnesses are not private optimizer bug claims. They are comparison-denominator errors that survive source removal, probe-family holdouts, engine-family stress, SQL executability, and two-engine visible-plan sanity checks. The missing distinctions are small and interpretable: placement repairs keyword collisions, optimizer layer repairs operator-only collisions, and the layer+placement basis separates all 498 headline witnesses across keyword, operator-only, and yes/no projections. OptSem-C turns coarse optimizer feature labels into replayable contract-denominator claims: each reported collision has public evidence, a reference-signature separation, and a retained-field repair.

PVLDB Reference Format:

Haoyi Zhang and Huaijin Ran. OptSem-C: Benchmarking Projection Precision in Public Query-Optimizer Contracts. PVLDB, 20(1): XXX-XXX, 2027.
doi:XX.XX/XXX.XX

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit <https://creativecommons.org/licenses/by-nc-nd/4.0/> to view a copy of this license. For any use beyond those covered by this license, obtain permission by

PVLDB Artifact Availability:

The source code, data, and/or other artifacts have been made available at <https://doi.org/10.5281/zenodo.21185236>. Archive SHA-256: 1e37facd df0c051fcf2e778389a86fe2208485f337c833378f4f09afa4bf3b6e.

1 INTRODUCTION

SQL deliberately hides physical execution choices. That abstraction is useful for applications, but it is too coarse for optimizer comparison. The same query can be accepted by several analytical engines while exposing different public behavior: a local predicate rewrite in one engine, connector-side execution in another, a service-side stage graph in a third, and only a logical explanation in a fourth. These differences determine what a benchmark author, system integrator, or database researcher may conclude about join ordering, pushdown, materialization, adaptive execution, and plan evidence before any deployment-specific performance experiment begins.

The practical precision loss is not cosmetic. A migration checklist may equate local filter rewrite with remote-source delegation; an engine-selection study may compare compile-time join enumeration with runtime repartitioning; a debugging report may treat an estimated plan node and a service profile stage as the same evidence. In audited probe P0009, a keyword view equates BigQuery federated SQL pushdown [16] and query-stage evidence [17] with Trino pushdown [34] and dynamic filtering [33] as pushdown+explain. Reference contracts instead separate connector-source filter delegation, stage evidence, aggregate/projection pushdown, and runtime filtering. The repaired statement must separate connector delegation from dynamic filtering, or disclose a collapsed pushdown denominator. OptSem-C tells such studies which public distinction they preserved and which repair fields make the comparison auditable.

Database research already has precise abstractions for the objects it chooses to study. Relational semantics starts from Codd's model [18]; Selinger-style optimizers separate access-path search from costing [65]; Volcano exposes rule-driven and parallel query processing [36]; the Volcano optimizer-generator account makes extensible search explicit [38]; and Cascades turns that line into a task-driven framework [37]. Chaudhuri surveys optimizer architecture [10]; Ioannidis surveys cost and cardinality assumptions [48]; and Steinbrunn et al. study join-order search heuristics [70]. Public optimizer behavior is usually compared with much coarser vocabularies. A cell labeled "pushdown" may mean predicate movement inside the engine, partition pruning, aggregation delegation, join delegation, limit delegation, or runtime filtering. A cell labeled

emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.

Proceedings of the VLDB Endowment, Vol. 20, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

“join” may refer to a logical operator, an estimated physical operator, a distributed exchange, or a runtime profile entry. A claim that two engines are “cost based” may hide whether statistics are local, connector-provided, unavailable, or revised during execution. These shortcuts are not harmless summarization: they can create projection-induced contract collisions (a coarse vocabulary declares equality among public contracts whose reference signatures differ).

OptSem-C treats this precision problem as a query-processing systems problem, not as prose cleanup. A *public optimizer contract* is the optimizer behavior an engine exposes through plan interfaces, tuning surfaces, and system descriptions. Documentation can support a contract that is not visible in a particular textual plan, and a plan tag can expose behavior without proving the private cost model. The object is therefore not a hidden implementation trace and not a latency measurement. OptSem-C represents each public statement as a guarded contract rule over query features. The rule records the action, the guard under which it applies, where the action is placed, which optimizer layer owns it, when the decision is made, how the behavior is observable, and whether the public contract requires, permits, forbids, or does not specify the action.

Research questions. We ask three finite, auditable questions. RQ1 asks whether public optimizer claims can be encoded as source-linked contracts and executable probes without becoming a latency benchmark. RQ2 asks how often comparison vocabularies that appear in public feature claims collapse distinct source-supported contracts. RQ3 asks which semantic fields eliminate those collisions, and whether the repair survives held-out sources, query-feature families, and engine-pair families.

The central result is a projection-kernel account of these contract collisions. Comparing optimizer contracts through a keyword or operator vocabulary applies a predeclared projection to reference signatures: the full public evidence payload inside the declared schema, not an oracle over private optimizer state. If reference signatures disagree but fall into the same projected class, the projection has declared an equivalence that source-supported public signatures do not support. Contract collisions are therefore denominator errors for the comparison itself; performance or cost measurements should be interpreted only after the compared behavior has been named. The new object is not a taxonomy. It is a finite comparison relation with executable counter-witnesses, exact repair certificates, and negative controls. OptSem-C fixes the contract schema, source manifest, projection portfolio, and motif crosswalk before the audit; execution checks never admit rules or choose repairs. It then computes reference and coarse classes from the same source-linked records and reports repair fields as certificates. Keyword projection declares 2,284 engine-probe equivalences: 2,030 have matching reference signatures and 254 are separated by reference signatures. Operator-only projection declares 2,268 equivalences, including 238 collisions. Restoring placement repairs keyword collisions; restoring optimizer layer repairs operator-only collisions; restoring layer and placement repairs all 498 headline witnesses within the admitted public-contract corpus. A field-only stress test over all 256 retained-field vocabularies leaves only 44 unsafe regions, all covered by concrete counter-witnesses; after excluding action-variant identifiers, no singleton semantic field is safe.

We also introduce OptSemBench-C, a covering benchmark for optimizer-contract behavior. Its denominator is not raw query

count. It is coverage of optimizer-relevant feature interactions: source boundary, connector capability, join shape, predicate class, aggregation, statistics need, reuse, distribution, adaptivity, and control surface. The generated probes are executable SQL representatives. The probe space is then cross-checked against DPccp-style join enumeration [59], JOB optimizer evidence [53], later JOB diagnostics [54], exact-cardinality testing [11], progressive optimization [57], mid-query reoptimization [51], adaptive query processing [24], and eddies [3]. We use this audit as a published-motif vocabulary stress test: it asks whether comparison words used around published workloads have concrete semantic requirements in our probe space, not whether the original studies made the same claim or measured the same performance outcome.

Contributions. C1 defines a public optimizer contract as a query-processing object separate from SQL result equivalence and runtime performance; Section 3 gives the guarded rule model and the state-join semantics. C2 turns comparison validity into a finite projection audit: reference signatures define the public evidence relation, projected signatures expose counter-witnesses, and repair certificates identify fields that make a vocabulary safe. C3 contributes OptSemBench-C as a denominator design for optimizer-contract comparisons across seven analytical SQL engines; Section 4 reports source-linked admission, executable probes, external-motif coverage, and zero-failure admission checks. C4 measures the projection kernels and repairs with exact negative controls, confidence intervals, source removal, probe-family holdouts, and engine-family stress; Section 5 reports 498 headline collision witnesses and a layer+placement basis that repairs the headline projections without using fine-grained variant labels.

Each central claim is tied to a reproducible record rather than to prose alone. The model section defines the state table and projection obligations; the corpus section exposes source identifiers, line spans, and digest prefixes; the evaluation tables connect SQL execution, motif coverage, collision counts, and repair certificates to regenerated outputs. The case studies then show how those aggregate certificates change comparison judgments.

2 PRECISION LOSS IN OPTIMIZER COMPARISONS

Figure 1 gives the smallest example. A feature matrix maps several public optimizer contracts into the same label. After the mapping, behaviors with different placement, action kind, and evidence layer become indistinguishable, so a study that treats the label as equality can compare non-equivalent public optimizer behavior.

Feature-name collision. A cell such as pushdown=yes may denote predicate movement inside an engine, partition pruning, connector-side predicate delegation, aggregation delegation, join delegation, limit pushdown, Top-N delegation, or runtime filtering. These actions affect different operators and have different guards, placements, decision times, and observable plan effects. Runtime and adaptivity distinctions are visible in Tukwila’s adaptive integration [49], dynamic query plans [19], and Spark SQL adaptive execution [2]. Rewrite and execution-layer distinctions appear in

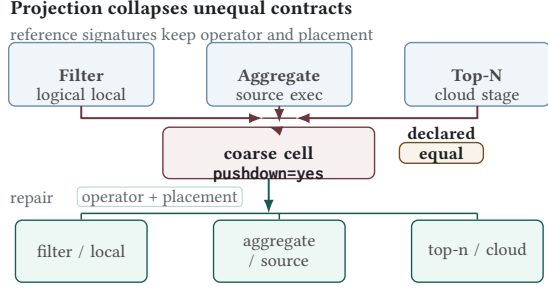


Figure 1: Projection loss in optimizer comparison. A coarse pushdown=yes label collapses unequal reference contracts; retaining operator and placement separates them.

outerjoin rewrites [35], compiled execution [60], Calcite’s adapter-centered optimizer framework [4], Orca’s modular optimizer architecture [69], and Dremel-style serving trees [58]. A single feature name can therefore erase exactly the fields a comparison needs.

Operator disappearance. A missing native plan node is not a reliable semantics. It may indicate source delegation, logical-plan observability, rewrite, stage-level aggregation, or runtime-only visibility. OptSem-C represents source-delegated execution as a positive contract action rather than as absence. This matters for modern analytical systems whose public surfaces combine Snowflake-style cloud-service stages [22], Redshift’s managed warehouse architecture [42], embedded DuckDB execution [62], vectorized MonetDB/X100 execution [6], distributed SQL services such as F1 [67], and dataflow-style execution such as Spark RDDs [75].

Architecture ranking. A local engine should not be ranked below a distributed engine merely because broadcast, shuffle, or remote-source delegation is not applicable. Conversely, a federated or cloud engine should not be credited with local optimizer behavior when the public contract exposes delegation or service-side stages. OptSem-C separates non-applicability, documented absence, documented optional behavior, and unspecified evidence.

Observable contract versus hidden implementation. The object of study is not what an engine could do internally. It is what the public contract supports as a comparison claim. This distinction is analogous to the separation between SQL equivalence and optimizer-contract equivalence. Cosette proves SQL equivalences by reducing query semantics to logic [15]; semiring-based decision procedures give another equivalence foundation [14]; and QED expands practical SQL equivalence checking [72]. Chandra and Merlin characterize conjunctive-query containment [9]; Aho et al. characterize equivalence among relational expressions [1]. OptSem-C asks a different question: whether the public optimizer behavior being compared exposes the same evidence layers and comparison obligations.

These precision losses imply a minimum payload for comparing public optimizer behavior: guard, canonical action, modality, placement, decision time, observability, version, and provenance. The rest of the paper formalizes this payload and measures what happens when a comparison vocabulary projects it away.

Table 1: Conflict-aware state join for contract support states (U=UNSPEC, N=MUST_NOT, C=CONFLICT).

$\sqcup$	U	MAY	MUST	N
U	U	MAY	MUST	N
MAY	MAY	MAY	MUST	C
MUST	MUST	MUST	MUST	C
N	N	C	C	N

3 CONTRACT FORMALIZATION AND NOTATION

A query probe is a SQL skeleton paired with a finite feature vector and validity constraints. Features describe optimizer-relevant structure: join shape and type, predicate class, aggregation, order/limit behavior, source boundary, connector capability, statistics need, reuse structure, distribution trigger, adaptivity trigger, and optimizer control surface. They are not workload-frequency estimates; they are coordinates on the public optimizer decision surface.

DEFINITION 1 (PUBLIC CONTRACT RULE). Let $\mathcal{A}$ be the finite domain of canonical optimizer actions. An action records operator class, action kind, optimizer layer, execution placement, decision time, observability, and variant. The state set $\mathbb{S}$ contains MUST, MAY, MUST_NOT, and UNSPEC. A public contract rule is $r = (\gamma, a, s, v, \epsilon)$, where γ is a guard over query features, $a \in \mathcal{A}$, $s \in \mathbb{S}$, v is the version or time window, and ϵ is evidence provenance. For engine e , C_e is the finite rule set, and $C_e(q) = \{r \in C_e \mid \gamma_r(\phi(q)) = \text{true}\}$ is the rule set applicable to probe q .

The state domain has two components that feature matrices usually conflate. Behaviorally, MUST requires an action, MUST_NOT forbids it, and both MAY and UNSPEC leave it possible. Evidentially, MAY is public support for optional behavior, while UNSPEC is lack of public support. Equivalently, a state is interpreted as (m, u, e) : required behavior, allowed behavior, and evidence support. This is why documented optional pushdown and undocumented implementation possibility are different public contracts.

For each engine-probe pair, OptSem-C constructs a contract map $M_{e,q} : \mathcal{A} \rightarrow \mathbb{S}$, defaulting every action to UNSPEC. Applicable rules update the map with a conflict-aware state join $\sqcup$. The join strengthens unspecified actions when public evidence is found, preserves compatible stronger evidence, and reports contradictions instead of averaging them. The evidence component follows database-provenance practice: semiring provenance attaches support to derived facts [40], why/where provenance distinguishes explanation locations [7], provenance surveys separate data from explanation [12], and annotated containment keeps evidence annotations visible in reasoning [39]. That separation motivates our refusal to turn missing public evidence into negative evidence.

DEFINITION 2 (STATE-JOIN ALGEBRA). The binary operator $\sqcup$ is the complete 4×4 partial join shown in Table 1. It is total over non-conflicting pairs and returns CONFLICT for pairs that simultaneously require and forbid, or support and forbid, the same canonical action. Let $x \leq y$ denote that $x \sqcup y = y$ for non-conflicting states. Then UNSPEC is the least informative state, while MUST and MUST_NOT are incomparable incompatible commitments.

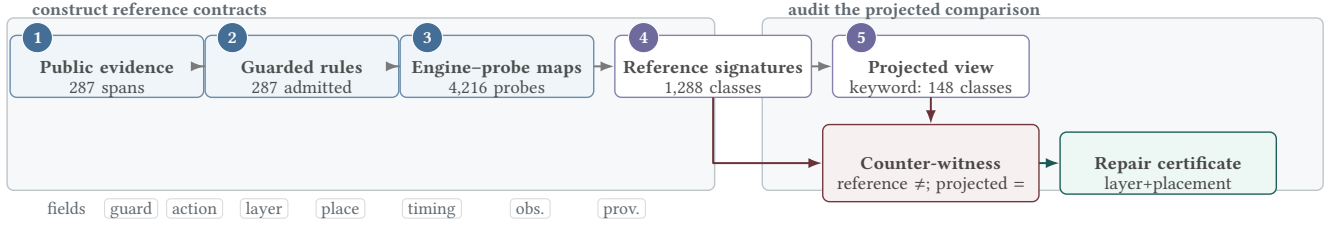


Figure 2: Contract-comparison pipeline. OptSem-C turns public evidence into reference signatures, exposes projection counter-witnesses, and reports finite repair certificates.

LEMMA 1 (MERGE INVARIANTS). *On non-conflicting states, $\sqcup$ is commutative, associative, idempotent, and monotone with respect to $\leq$. Therefore a contract map obtained by joining applicable rules is independent of rule order.*

PROOF. The properties follow by inspection of the finite table. Idempotence holds because every diagonal entry is the input state. Commutativity holds because the table is symmetric. Associativity and monotonicity are checked over the finite non-conflicting triples; the only excluded triples are exactly those containing incompatible positive and negative commitments. Lifting the pointwise operator to maps preserves the properties for every action independently. $\square$

Let $W \subseteq \mathcal{A}$ be an abstract optimizer-behavior world. A map M denotes the world set $\llbracket M \rrbracket = \{W \mid M(a) = \text{MUST} \Rightarrow a \in W, M(a) = \text{MUST_NOT} \Rightarrow a \notin W\}$. The evidential support set is $\text{Supp}(M) = \{a \in \mathcal{A} \mid M(a) \in \{\text{MUST}, \text{MAY}, \text{MUST_NOT}\}\}$, equivalently the actions where $M(a) \neq \text{UNSPEC}$. Since maps default to UNSPEC, every supported non-default entry arises from an admitted public rule. Behavioral containment and evidential support are intentionally separate: two maps can allow the same worlds while differing on whether an action is publicly documented.

DEFINITION 3 (REFERENCE SIGNATURES, KERNELS, AND PROJECTIONS). *The reference evidence-supported signature of M is $S(M) = \{(a, M(a)) \mid a \in \text{Supp}(M)\}$. Reference equivalence is equality of signatures over the full public evidence atom schema. This reference is exact only relative to the admitted public evidence and declared atom fields; it is not a complete private optimizer trace. A projection is a declared map $\pi : \mathcal{A} \times \mathbb{S} \rightarrow \mathcal{B}$ from full evidence atoms to an arbitrary comparison vocabulary $\mathcal{B}$, such as keyword, yes/no, or operator-only labels. The projected signature is the finite image $S_\pi(M) = \{\pi(a, M(a)) \mid a \in \text{Supp}(M)\}$. The projection kernel is the equivalence relation $\mathcal{K}_\pi$ where $M_i \mathcal{K}_\pi M_j$ iff $S_\pi(M_i) = S_\pi(M_j)$. A projection-induced contract collision is a pair in $\mathcal{K}_\pi$ whose reference signatures differ.*

For quantitative comparison, exact compatibility is the Jaccard overlap $|S(M_i) \cap S(M_j)| / |S(M_i) \cup S(M_j)|$, and projected compatibility applies the same formula to S_π . The denominator is evidence-supported: undocumented actions do not count as evidence of either equality or inequality.

LEMMA 2 (PUBLIC-CONTRACT SEPARATION). *If $S(M_i) \neq S(M_j)$, then the maps differ in public-contract denotation: either $\llbracket M_i \rrbracket \neq$*

$\llbracket M_j \rrbracket$, or their evidence-supported action states differ for at least one action in $\text{Supp}(M_i) \cup \text{Supp}(M_j)$.

PROOF. If reference signatures differ, some supported action has a state in one map that is absent or different in the other. If the difference changes a MUST or MUST_NOT requirement, the world-set constraints differ. Otherwise the behavioral world set may remain equal, but the public evidence state differs, for example MAY versus UNSPEC. In both cases the denotation differs because an OptSem-C contract is the pair of behavioral worlds and evidence-supported states. $\square$

For finite feature and action domains, constructing $M_{e,q}$ from finite C_e and q is a direct finite evaluation: each guard is a Boolean formula over the probe feature vector, and each applicable rule performs one lookup in the state-join table.

OBSERVATION (PROJECTION-KERNEL COLLISION). *For any projection π , a projection-induced contract collision is exactly the nontrivial part of the kernel $\mathcal{K}_\pi$: M_i and M_j collide iff $S(M_i) \neq S(M_j)$ and $M_i \mathcal{K}_\pi M_j$. If π is non-injective on evidence atoms, there exist contracts with such a collision witness.*

PROOF. The first statement unfolds the definitions: $\mathcal{K}_\pi$ is equality after projection, while exact non-equivalence is $S(M_i) \neq S(M_j)$. For the non-injective case, take distinct evidence atoms $(a_1, s_1) \neq (a_2, s_2)$ with $\pi(a_1, s_1) = \pi(a_2, s_2)$. Construct M_1 so that $S(M_1) = \{(a_1, s_1)\}$ and M_2 so that $S(M_2) = \{(a_2, s_2)\}$, with all other actions UNSPEC. These minimal contracts are well formed because action states are assigned independently. Then $S(M_1) \neq S(M_2)$, but $S_\pi(M_1) = S_\pi(M_2)$ is the same singleton, so (M_1, M_2) is a collision witness. $\square$

DEFINITION 4 (PROJECTION INFORMATION PROFILE). *For a finite set of observed maps Y , let $H(S)$ be the empirical Shannon entropy of reference signatures and $H(S_\pi)$ the entropy of projected signatures. The entropy-retention ratio $\rho_\pi = H(S_\pi)/H(S)$ measures distinguishability retained by the comparison vocabulary. The projected collision count is the number of projected-signature classes containing more than one reference signature class.*

LEMMA 3 (KERNEL INFLATION). *If a projection declares E_π equivalent pairs and reference comparison declares E over the same pair set, then E_π/E is the pairwise inflation factor induced by the kernel. Every collision witness contributes to this inflation, and a zero-collision strengthened projection must have $E_\pi = E$.*

PROOF. Reference equivalence implies projected equivalence only when the projection is a function of the full atom payload, so $E_\pi \geq E$. The additional pairs counted by E_π are exactly the unequal reference signatures that collapse under π , which are collision witnesses. If there are none, no additional pairs are counted and the two denominators agree. $\square$

OBSERVATION (EVIDENCE REFINEMENT). *Adding a non-conflicting evidence-backed rule cannot silently weaken a public contract: it behaviorally refines possible worlds, evidentially refines support, or both.*

The remaining notation is the payoff of the model: it turns an informal vocabulary dispute into a finite certificate check. Let F be the semantic-field universe and let π_R append field values in $R \subseteq F$ to the baseline projection. For a finite comparison set X , define $X_{\pi_R} = \{(M_i, M_j) \in X \mid S(M_i) \neq S(M_j) \wedge M_i \mathcal{K}_{\pi_R} M_j\}$. A repair R is safe for π on X when $X_{\pi_R} = \emptyset$; otherwise any pair in X_{π_R} is a counter-witness. Since adding retained fields can split projected classes but cannot merge already separated classes, safe sets are upward closed and unsafe sets are downward closed. For each witness w , the separator family $\mathcal{D}_w = \{R \subseteq F \mid M_i \not\mathcal{K}_{\pi_R} M_j\}$ is upward closed, so universal repair is the finite intersection problem $R \in \bigcap_{w \in X_\pi} \mathcal{D}_w$. The reported certificates therefore consist of all repaired witnesses plus counter-witnesses for smaller field sets; exhaustive enumeration over $2^{|F|}$ retained-field subsets is complete for the declared field universe.

PROPOSITION 1 (MINIMUM REPAIR AS HITTING SET). *For finite semantic fields F and collision witnesses X_π , deciding whether there is a universal repair of size at most k is NP-complete in $|F| + |X_\pi|$. For OptSem-C’s fixed field universe, minimum repairs are exactly enumerable.*

PROOF. Membership is finite checking. Hardness reduces HITTING SET: create one witness per set $H_i \subseteq F$ whose two contracts differ exactly on H_i ; a repair separates that witness iff it intersects H_i . $\square$

The semantics makes the comparison payload explicit. A user comparing only operator names chooses a projection that drops placement, decision time, variant, and observability. A feature checklist chooses an even coarser projection that can drop operator class itself. OptSem-C does not forbid those projections; it computes exactly which kernel collisions they induce, which grounded counter-witnesses remain, and which query-processing fields repair the comparison.

4 BENCHMARK AND GROUNDED CORPUS

OptSemBench-C is a behavior-contract benchmark: it generates probes that cover optimizer-relevant feature interactions and measures which public contracts those probes activate. It deliberately does not rank engines by latency. This choice follows the benchmark principle that the metric and denominator must match the object of study. CardEst asks whether cardinality estimates improve DBMS plan choice [43]; CardBench stresses learned cardinality estimators in relational settings [13]; Learned Cardinalities models correlated joins [52]; NeuroCard models probabilistic cardinalities [74]; DeepDB learns database distributions [44]; FactorJoin specializes

join-cardinality estimation [73]; Neo learns plan choices [56]; and Bao evaluates learned hinting behavior [55]. An optimizer-contract benchmark must instead expose semantic traps in the comparison vocabulary.

The contract schema is fixed before source extraction, witness counting, and repair search. Its fields come from prior optimizer architecture rather than from the measured collisions. Operator/action and layer follow System R [65], Volcano [36], Cascades [37], Calcite [4], and Orca [69]. Placement follows Garlic federation [8], BigDAWG polystores [25], Dremel serving trees [58], and Snowflake cloud execution [22]. Decision time follows progressive optimization [57], mid-query reoptimization [51], adaptive query processing [24], eddies [3], and Spark adaptive execution [2]. Observability follows PostgreSQL EXPLAIN [41], BigQuery query plans [17], Snowflake EXPLAIN [47], and provenance-style evidence separation [40]. Layer and placement later emerge as repair fields, but enter as predeclared dimensions.

Let $F = F_1 \times \dots \times F_n$ be the finite feature domain and Ψ the validity constraints. A target interaction is a valid partial assignment over a feature subset. The schema and feature domain are fixed before witness counting and derive from standard query-processing constructs and cited public optimizer surfaces, not from witnesses or motifs. The benchmark targets global pairwise coverage plus selected three-way interactions for source boundary, connector capability, operator class, statistics need, join shape, adaptivity trigger, and distribution trigger. A greedy generator selects renderable full assignments until all renderable target interactions are covered. The generator has two responsibilities. First, it separates benchmark adequacy from any particular system: adequacy is measured against the feature-interaction universe, not against runtime results. Second, it produces probes that are diagnostic rather than workload-frequency estimates. A probe is useful when it activates a contract guard or separates two contract signatures.

OBSERVATION (COVERAGE). *If every valid target interaction has at least one renderable full assignment, the generator terminates with a probe set covering every renderable target interaction.*

PROOF. The target universe is finite. Each selected assignment covers at least one uncovered interaction. Thus the number of uncovered interactions decreases monotonically until it reaches zero. $\square$

The main empirical corpus contains only 287 source-linked grounded rules. Each rule records a public evidence span, source URL, line range, digest, and source class. Claims without grounded evidence, expressible guards, or public observability are outside the finite denominator rather than negative evidence. Thus the paper claims coverage of the declared public-contract denominator, not a census of private optimizer rules. The corpus covers plan observability, pushdown, join search, distribution, materialization, runtime adaptivity, statistics, and optimizer control surfaces. Every grounded rule is triggered by at least one generated probe, and contract merge produces no conflicts.

The source set is fixed before witness counting. We admit an engine only when public sources provide: a comparable plan or

Table 2: Source-to-rule traceability for representative contracts. Each row binds a rule state/action to a public source span, digest prefix, and paraphrase used by the replay manifest.

Contract rule			Auditable evidence	
Engine	State	Action	Span/digest	Source-supported paraphrase
PostgreSQL [41]	MAY	Plan/observe/local	PG-EXPLAIN-001 bfb47e66 L46–L56	EXPLAIN exposes estimated startup and total cost, output rows, row width, and planner-cost units.
BigQuery [17]	MAY	Exchange/adapt/cloud	BIGQUERY-PLAN-001 cc7b206a L990–L992	The query plan may be modified during execution by adding repartition stages.
Snowflake [47]	MAY	Plan/observe/cloud	SNOWFLAKE-EXPLAIN-001 93a61359 L150–L154	EXPLAIN compiles a statement and returns a logical execution plan rather than executing it.
ClickHouse [46]	MAY	Join/adapt/local	CLICKHOUSE-JOINING-001 a1f757e2 L313	With automatic join-algorithm selection, execution can switch from hash join to partial merge join.
DuckDB [30]	MAY	Materialize/inline/local	DUCK-WITH-001 2a60c8ab L482–L487	CTE inlining is guarded by single use, absence of volatility, no grouped aggregation, and inlining hints.
Spark SQL [28]	MAY	Plan/adapt/distributed	SPARK-PERF-001 7e5e2b00 L129–L133	Adaptive query execution uses runtime statistics and can re-optimize a query while it is running.
Trino [34]	MUST_NOT	Agg./pushdown/source	TRINO-PUSH-001 cc177219 L168–L176	Aggregation pushdown does not support aggregate calls containing expressions.

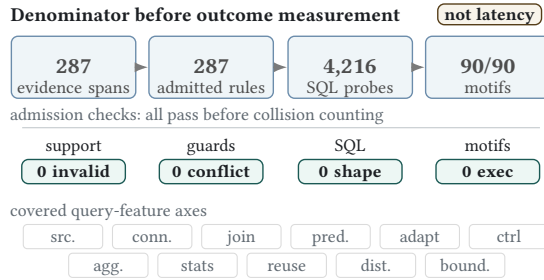


Figure 3: Corpus, benchmark, and execution denominators. The dashboard fixes evidence, SQL probes, admission checks, and motif coverage before collision rates are reported.

contract surface, enough detail to extract an action and applicability guard, and stable analytical-SQL behavior rather than a private vendor note. PostgreSQL exposes planning and EXPLAIN surfaces [41]; Trino documents cost-based optimization [32], connector pushdown [34], and dynamic filtering [33]; BigQuery publishes query-stage plan explanations [17] and federated SQL pushdown limits [16]; Spark documents adaptive execution [28]; Snowflake documents logical explanation [47]; DuckDB documents CTE materialization controls [30]; and ClickHouse documents join algorithms and optimization guidance [46]. Closed commercial optimizers without public rule evidence and framework-only rule catalogs without a comparable engine surface are excluded from the collision denominator; systems such as Calcite still inform the schema and related-work positioning. The sources are not performance measurements. They provide the public contracts that the framework formalizes.

The source-linked denominator spans seven engines: PostgreSQL contributes 80 admitted rules, Trino 61, BigQuery 37, Spark SQL 36, ClickHouse and Snowflake 26 each, and DuckDB 21; the largest single source supports 46 rules. This design separates admission, probing, and execution validation. Public sources admit rules into the contract denominator; generated probes test guard reachability and SQL executability; DuckDB and PostgreSQL runs check two

open execution surfaces for the generated SQL. A DuckDB source span can admit a DuckDB public contract, but a later DuckDB execution is recorded only as SQL execution and visible-plan sanity evidence. It is not a vote that the source was true, it does not admit rules for any engine, and it is never used to choose a repair field. Conversely, engines without local execution still participate in collision measurements only through source-linked public contracts, source leave-one-out checks, and side-balanced witness support.

The grounded corpus is intentionally conservative. A rule is admitted only when the evidence span supports an optimizer action, a guard or applicability condition, and an observability or placement interpretation. Vague performance advice is excluded from the main corpus. This policy reduces corpus size but strengthens the central claim: the evaluation is about contracts that are publicly supported, not about inferred implementation behavior.

Contract extraction and validation checks. Each admitted rule records the engine, version scope, source identifier, digest, action fields, guard, and observability surface. Human coding maps public evidence spans into the fixed schema; automated checks validate schema consistency, source linkage, guard reachability, trigger coverage, merge conflicts, SQL executability, and table-source consistency. This is a source-line admission protocol, not formal verification or recoding agreement: every accepted row is inspectable, and excluded rows are outside the declared denominator. Table 2 gives representative spans; the replay materials carry the complete rule-to-source manifest, source digests, claim-evidence graph, and regenerated table manifests. The field schema and projection portfolio are replay inputs fixed before collision counting. Layer and placement are members of this predeclared field universe; exhaustive enumeration discovers the frontier, and regenerated kernel counts and repair tables validate layer+placement as a frozen semantic basis rather than a field pair chosen after inspecting one witness family. Documentation admits the public claim; EXPLAIN, profile, or stage surfaces bind exposed plan evidence when available; SQL execution validates the probe denominator rather than private optimizer truth. A counted witness requires two source-supported reference signatures, projection equality under the declared vocabulary, and separation by the reported repair field.

The denominator dashboard in Table 3 checks that this conservatism does not create unreachable contracts or syntactic probes. A finite-disjunction guard semantics evaluates each admitted rule against the generated probe denominator. All 287 grounded rules have at least one triggering probe; 252 depend on query features rather than global engine labels; and no guard uses a feature dimension or value outside the benchmark domain.

Coverage is reported over two denominators. The first is benchmark coverage: every targeted renderable valid feature interaction is covered by at least one generated probe. The second is contract triggering: every grounded rule is applicable to at least one probe. Together these denominators make the empirical scope explicit. A rule that cannot be triggered by any probe would indicate a benchmark gap; a feature interaction that cannot be rendered is excluded from the coverage denominator and reported as a modeling constraint.

The benchmark also exposes executable SQL shapes. Each generated probe has a normalized SQL skeleton and shape checks for joins, filters, grouping, ordering, limits, and reuse structure. The full 4,216-probe corpus is accompanied by a compact 48-probe motif-cover prefix. The real-engine motif execution set uses the deterministic union of this motif cover and residual feature-axis maxima; it is not a hand-picked success set, and it is not a replacement for the full evaluation.

A common failure mode in generated benchmarks is to produce feature-complete but non-executable query text. OptSemBench-C therefore validates every SQL skeleton on a deterministic relational catalog with local, remote, and cross-source namespaces. This validation is not a performance baseline and is not exhaustive contract-trigger proof. Its role is stricter and more basic: every generated probe must be parseable, plannable, executable, and shape-consistent with its feature vector. Table 3 reports the validation result: all 4,216 generated probes obtain a plan and execute, with zero SQL-shape failures. Repeating the full corpus across three deterministic catalog populations yields 12,648 successful probe-catalog executions, and DuckDB/PostgreSQL checks add 8,432 full-corpus engine-probe executions plus 142 motif-representative executions with zero execution failures.

Contract validation is layered. Guard reachability says that each public rule has a generated probe satisfying its admitted guard; SQL execution says that the denominator is not syntactic fiction. Visible-plan sanity checks on DuckDB and PostgreSQL expose expected plan tokens at mean rates 0.705 and 0.796 over the full corpus, and 0.756 and 0.819 on motif representatives. These checks do not reverse-engineer private optimizers, prove every documented behavior fires on every cost-model instance, or cross-validate a source by running the same engine. A counted witness still requires source-supported reference signatures, guard reachability, projection equality, and repair separation; runtime plan satisfaction is a separate SQL-denominator validation layer.

The same fixed probes support two budgeted evaluation orders. The cover order is the generator’s feature-interaction order. The diagnostic order greedily maximizes newly triggered grounded rules without adding any new probe. Table 4 reports both. The cover order triggers all grounded rules within 100 probes; the diagnostic order triggers 286 of 287 rules within 50 probes and all rules within

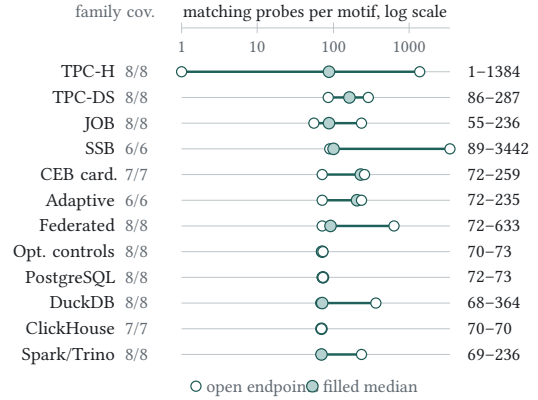


Figure 4: Published optimizer-motif denominator. The log-scale range plot shows min/median/max matching probes per family; all 90 requirements are executable. The motif-family sources are cited individually in the surrounding crosswalk paragraph.

100. Random subsets of 100 probes trigger 81% of rules on average and 85% at the 95th percentile. Thus OptSemBench-C is not merely large enough to cover the surface; it also contains compact diagnostic prefixes that expose almost all grounded contracts.

A benchmark for public optimizer contracts must connect to existing database workloads without becoming another latency benchmark. We therefore add a crosswalk from published optimizer and benchmark motifs to OptSemBench-C feature requirements, covering JOB join-order sensitivity [53], POLAR-style join-order structure [50], TPC-H [21], TPC-DS [20], SSB [61], LDBC [26], Wisconsin [5], AS3AP [71], ClickBench [45], and LakeBench [23]. Figure 4 reports the full executable denominator: 90 declared requirements across 12 motif families, all covered. Table 5 lists five compact anchors; Table 6 gives the 12-family difficulty distribution. The selected probes are feature-interaction representatives, not replacements for JOB or TPC-H queries; the crosswalk validates optimizer-decision coverage rather than workload runtime behavior. It remains a frozen audit over optimizer-decision surfaces, not copied benchmark SQL or a latency claim; motifs outside the declared feature domain would be denominator misses. Because motifs are checked after the schema is fixed and before repair fields are used, this is a non-circular denominator check: a counted collision still requires two source-supported signatures, projection equality, side-balanced support, and repair separation. We claim contract-denominator adequacy and published-motif coverage, not third-party workload-performance reproduction.

The motif-difficulty view prevents a misleading denominator. Some motifs are broad and match many probes, such as star-schema group-by shapes. Others are sparse, such as a TPC-H conjunctive-filter motif that has one matching rendered probe under the current validity constraints. Reporting both coverage and matching-probe counts makes benchmark adequacy inspectable: coverage says the motif is present, while difficulty says how concentrated or redundant that coverage is. The executable motif suite adds a second denominator check: for each published motif, the system selects

Table 3: Benchmark denominator and validation checks across corpus, SQL, real-engine SQL execution, and motif layers. Bold marks complete coverage or zero-failure checks; execution rows validate query construction, not contract truth.

Layer	Check	Value	Scope	Interpretation
Corpus	grounded rules	287	26 sources	finite public-contract relation
Corpus	probe-supported rules	287/287	median support 102	every admitted rule is triggerable
Corpus	non-global guards	252	3 containments	query-conditional, audited overlaps
Guard schema	invalid dimensions	0	fixed feature domain	no out-of-domain guard values
SQL corpus	generated probes	4,216	digest df107f018e37	full executable denominator
SQL corpus	motif-cover prefix	48	human inspection	compact replay slice, not cherry-picking
SQL execution	planned/executed probes	4,216/4,216	91 distinct plans	every generated probe plans and runs
SQL execution	shape/exec failures	0/0	18,448 rows	no SQL-shape or runtime failures
Real engines	full DuckDB/PostgreSQL	8,432/8,432	tags .705/.796	SQL validation, not source truth
Real engines	motif DuckDB/PostgreSQL	142/142	tags .756/.819	budgeted open-engine sanity
Published motifs	covered requirements	90/90	12 families	every declared motif has executable probe support

Table 4: Probe budgets for triggering grounded contract rules; random columns summarize subset baselines. Bold marks complete trigger coverage.

Budget	Cover	Diagnostic	Random mean	Random 5–95
50	.672	.997	.707	[.627,.767]
100	1.000	1.000	.811	[.770,.850]
250	1.000	1.000	.890	[.864,.913]
500	1.000	1.000	.920	[.899,.941]
1000	1.000	1.000	.951	[.934,.969]
2000	1.000	1.000	.973	[.958,.986]
4216	1.000	1.000	1.000	[1.000,1.000]

Table 5: External vocabulary stress test. Rows are literature/workload motif families, not copied SQL or performance claims; every declared optimizer-surface requirement has executable probe support.

Anchor	Surface	Req.	Cov.
JOB	join-order search	8	8
Exact-card.	exact-cardinality testing	9	9
CardEst	plan-quality sensitivity	8	8
Adaptive	runtime replanning	9	9
Lake	cross-source joinability	7	7

Table 6: Matching-probe difficulty across published optimizer motifs. Underlining marks the sparsest and broadest motif coverage extremes; motif-family sources are cited in the crosswalk paragraph.

Suite	Motifs	Min	Median	Max
TPC-H	8	<u>1</u>	87.0	1384
TPC-DS	8	86	164.5	287
JOB	8	55	87.0	236
SSB	6	89	101.0	<u>3442</u>
CardEst	7	72	228.0	259
Adaptive	6	72	205.5	235
Federated	8	72	91.5	633
Controls	8	70	72.0	73
PostgreSQL	8	72	73.0	73
DuckDB	8	68	72.0	364
ClickHouse	7	70	70.0	70
Spark/Trino	8	69	70.0	236

one satisfying probe and validates the generated SQL on the deterministic catalog, yielding 90 validated motif representatives and

383 result rows. Thus the motif denominator is not a list of names; it is a set of optimizer-decision requirements with concrete SQL representatives. Engine-plan validation is a separate experiment because the motif suite itself is a denominator check rather than a claim that published benchmark queries and generated probes induce identical plans.

5 EVALUATION

The evaluation asks four questions: how often coarse comparisons collapse unequal reference signatures, which fields repair the collisions, whether collisions reflect recognizable query-processing mechanisms rather than construction effects, and how much distinguishability each vocabulary retains. The schema, projections, and repair search are fixed before derived tables are generated: reference signatures define the public evidence partition; each coarse vocabulary declares candidate equivalence classes; repairs are evaluated over all counted witnesses. The table column “false” is contract-level: it means that a declared vocabulary collapses unequal public optimizer-contract signatures. Such a collision makes the comparison denominator semantically invalid before any latency, plan-quality, or cost claim is interpreted.

The baselines are executable comparison vocabularies, not DBMS-testing tools. Cosette [15] and QED [72] verify SQL equivalence; SQLsmith [66], SQLancer/PQS [64], and NoREC [63] find DBMS bugs or wrong answers. Neither family defines equality for public optimizer-contract denominators. To keep the projection portfolio practice-aligned, we audit the 26 public source surfaces used by the corpus: 26 expose keyword or feature-name labels, 12 expose enablement or yes/no controls, 26 expose operator-family labels, and none exposes the complete reference contract payload used by OptSem-C. Keyword, yes/no, and operator projections mirror public comparison surfaces; reference comparison is the negative control; one-field projections test whether a single optimizer dimension can stand in for contract semantics; strengthened projections restore fields to localize missing information. All projections are fixed before witness counting and evaluated over the same source-linked maps. The question is how many witnesses are lost if equality is decided at that vocabulary, not whether every cited study made that exact projection. Feature-name labels come from Trino pushdown documentation [34] and Presto documentation [31]; workload labels use TPC-H [21] and TPC-DS [20]; plan-quality controls follow JOB [53], later JOB diagnostics [54], and exact-cardinality testing [11].

Table 7: Projection-induced contract collisions and minimum repair fields for headline vocabularies. Bold marks certified repair fields; underlining marks the two headline unsafe baselines emphasized in the text.

Projection	Eq.	False	Cond.	Repair field(s)
Keyword	2,284	<u>254</u>	<u>11.1%</u>	placement
Yes/No	2,036	6	0.3%	layer
Operator-only	2,268	<u>238</u>	<u>10.5%</u>	layer

Table 8: Collision severity under lossy projections, measured by reference-signature distance and atom gaps. Bold marks the zero-collision control; underlining marks the largest un-repaired unsafe residual evidence.

Projection	False	Dist.	Mean Δ	Max Δ
Keyword	254	1.00	9.09	17
Yes/No	6	1.00	3.00	3
Operator	238	1.00	5.84	14
Placement	10,702	1.00	21.29	53
State	<u>23,593</u>	1.00	13.06	<u>64</u>
Surface	0	0.00	0.00	0

Coarse comparison induces collisions. Table 7 reports the main measurement. The keyword projection declares 2,284 equivalences: 2,030 pairs agree under reference public-contract signatures and 254 pairs disagree. The operator-only projection declares 2,268 equivalences, including 238 pairs whose reference signatures disagree. These errors are conditional on the baseline declaring equivalence, which is the decision faced by a user of a feature matrix. The rates are not sampling artifacts: Wilson and probe-cluster bootstrap intervals keep keyword at roughly 10–12% and operator-only at roughly 9–12%, while the strict reference projection has zero false equivalences. The collision count remains nonzero for keyword and operator-only projections after removing any single public source.

Counts alone understate the collision. Table 8 measures how far apart the reference contracts are after a projection has declared them equal. For the headline projections, every collision pair has reference-signature Jaccard distance 1.0: the projected comparison has equated disjoint public contract signatures rather than near duplicates. Keyword collision pairs differ by 9.09 evidence atoms on average, and operator-only collision pairs differ by 5.84 atoms. The strengthened surface projection is the control: it returns to the reference-equivalence denominator and has zero projection-induced collision.

Table 9 is the vocabulary ablation. It evaluates exact negative controls, common feature matrices, one-field adversarial vocabularies, and strengthened high-information baselines. The exact-contract baseline produces no collisions. Keyword, kind-only, and operator-only comparisons induce collisions at similar rates, so the result is not tied to one synonym list. One-field baselines such as placement-only, decision-time-only, and state-only are much worse, showing that a single optimizer dimension is not a safe substitute for contract semantics. By contrast, the strengthened semantic-surface baseline has zero collisions, which localizes the missing fields rather than merely declaring all abstractions bad.

Table 9: Executable projection baselines. Bold zeros are exact or strengthened controls; underlining marks the worst one-field projection, exposing the shown range.

Baseline	Eq.	False	Rate
Exact	2,030	0	0.0%
Keyword	2,284	254	11.1%
Op/action	2,036	6	0.3%
Operator	2,268	238	10.5%
Kind-only	2,282	252	11.0%
Layer-only	2,479	449	18.1%
Place-only	12,732	10,702	84.1%
State-only	25,623	<u>23,593</u>	<u>92.1%</u>
Surface	2,030	0	0.0%

Projection kernels are measurable

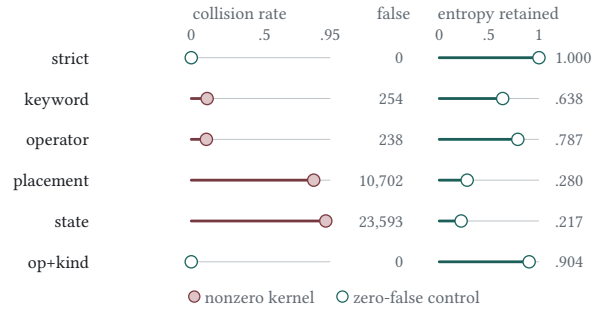


Figure 5: Projection information profile. Rose bars mark nonzero projection kernels; false-pair counts give witness denominators, while teal controls keep zero false pairs.

Projection kernels are measurable objects. Figure 5 reports the finite information profile of representative projections. Reference signatures form 1,288 observed classes. Keyword projection compresses them into 148 projected classes, 81 of which contain multiple reference classes, retaining 64% of empirical signature entropy and inflating declared equivalence by 12.5%. Operator-only projection retains more entropy but still leaves 191 colliding projected classes and 238 pairwise collisions. Placement-only and state-only baselines demonstrate the opposite failure mode: a single field creates very large kernels. The strengthened surface projection still has projected collisions at the signature-class level, but it does not produce any pairwise collision over the engine-probe comparison set. Thus the kernel is finite, populated, and repairable in the observed comparison relation.

Resolution-lattice stress test. The information profile asks how much a named projection compresses reference signatures. Table 10 asks a stricter question: if a comparison is allowed to retain any subset of public optimizer fields, which retained-field vocabularies are safe? The test enumerates all 256 subsets of the eight evidence-atom fields and evaluates them over the same 88,536 engine-probe comparisons. The empty vocabulary declares 59,546 unsupported equivalences. Every singleton semantic field is unsafe: operator alone leaves 238 collisions, kind alone leaves 252, layer alone leaves 449, and placement alone leaves 10,702. Excluding action-variant identifiers, the semantic field lattice has 128 subsets, 44 unsafe regions, and minimum safe field count two; the

Table 10: Retained-field lattice for projection repair. Bold marks safe zero-collision vocabularies; underlining marks the largest unsafe collision count.

Fields	R	Classes	Ent.	Eq.	False
None	0	1	0.000	61,576	<u>59,546</u>
Operator	1	387	0.787	2,268	238
Action kind	1	183	0.661	2,282	252
Layer	1	52	0.552	2,479	449
Placement	1	9	0.280	12,732	10,702
Operator+layer	2	591	0.850	2,030	0
Kind+placement	2	214	0.680	2,030	0
Surface	5	820	0.904	2,030	0
All sem.	7	947	0.934	2,030	0

Table 11: Statistical and source-removal robustness of projection collisions. Wilson and Boot. are 95% intervals for the conditional false-equivalence rate; LOO removes one public source at a time.

Proj.	False	Micro	Wilson	Boot.	Pair	LOO
Keyword	254	0.111	[.099,.125]	[.098,.124]	0.402	[.017,.144]
Yes/No	6	0.003	[.001,.006]	[.001,.006]	0.003	[.000,.052]
Operator	238	0.105	[.093,.118]	[.094,.121]	0.290	[.032,.148]

two minimum safe no-variant vocabularies are operator+layer and kind+placement. A compact set of ten concrete engine-probe witnesses covers all unsafe retained-field regions, so the frontier is not driven by one example row. Deriving the frontier antichains gives 14 minimal safe no-variant vocabularies and eight maximal unsafe vocabularies; the monotone frontier certificate shows that every safe retained-field vocabulary contains one of the minimal safe elements and every unsafe vocabulary is contained in a maximal unsafe element.

The robustness view in Table 11 checks that the headline result is not an effect of a single denominator. The micro rate conditions on the baseline declaring equivalence; Wilson and probe-cluster bootstrap intervals quantify uncertainty over declared equivalences and probe clusters; the pair-macro rate averages within engine pairs; and the leave-one-source range removes all rules supported by one public source. Keyword and operator-only collisions remain visible across these views, with leave-one-source conditional-rate ranges [0.017, 0.144] and [0.032, 0.148] respectively.

The evidence side is cross-checked without retraining on the headline witnesses. Every collision witness has at least two public source records and side-balanced support on both compared signatures. Source leave-one-out changes the public-evidence denominator, feature-family holdout learns repairs away from one query slice, and engine-family stress changes the compared engine pairs; none creates an unresolved witness after the layer+placement repair. These checks are not meant to make a perfect repair surprising by itself; they show that the same small basis survives independent changes to source support, query-feature family, and engine-pair family after the schema is fixed.

The 287-rule corpus is a finite public-contract sample, not a census of optimizer internals, so the remaining checks ask whether the observed kernel is a one-source or one-shape accident. Keyword and operator-only collisions survive every leave-one-source

Table 12: Dispersion of collision witnesses across probes, engine pairs, and feature values. Larger support columns show broader exposure; lower Max means less concentration. Bold marks the largest observed support in each support column.

Projection	Wit.	Probes	Pairs	Values	Max
Keyword	254	254	3	92	1
Operator-only	238	238	2	75	1
Yes/No	6	6	1	15	1

Table 13: Finite-comparison scalability and dispersion. False and Drift are lower-better; Checks/s is higher-better. Bold marks per-column best values for directional columns. Probes/Axes/Fam. describe where witnesses appear.

Proj.	False	Probes	Axes	Fam.	Checks/s	Drift
strict	0	–	–	0/6	221,440	0
keyword	254	254	9	3/6	279,920	0
operator-only	238	238	8	2/6	313,012	0
surface	0	–	–	0/6	240,801	0

removal; the 498 headline witnesses occupy 486 distinct probes; and the largest source affects 46 rules, 4,216 maps, and 14.3% of rule-probe rows. Table 12 further shows that witnesses touch every feature dimension and cover 92 and 75 observed feature values for keyword and operator-only projections. The witnesses are therefore dispersed projection-kernel evidence rather than duplicated rows.

Cost and scalability. Table 13 adds stress views. The finite audit executes all 88,536 engine-probe pair checks per projection, so the kernel is not sampled. The full audits complete in seconds with throughput above 50,000 comparisons per second; an 8× lift reaches 708,288 comparisons per projection with throughput above 200,000 comparisons per second and prefix-scaling $R^2 \geq 0.98$. Streaming maintenance has zero drift and uses 88,536 work units rather than the 186.7 million implied by recomputing every prefix. A source refresh is local: the largest public source affects 46 rules, 4,216 maps, and 14.3% of rule-probe map rows. The table also ties scaling to dispersion and family stratification: keyword collisions span 254 probes, nine feature axes, and three of six pair-family regions; operator-only spans 238 probes, eight axes, and two regions.

The probe-subsample view checks that the collision result is not an effect of using the full generated benchmark. At 10% and 50% probe subsamples, keyword and operator-only projections retain at least one collision in every trial, with median conditional rates close to the full benchmark. The weaker yes/no projection is sparse at 10%, but becomes nonzero in every trial by 50%.

The mutation-suite view is adversarial: it expands the comparison vocabulary around the headline baselines rather than only evaluating the three projections used in the abstract. Reference signatures remain a zero-false negative control, one-field projections produce much larger kernels, and strengthened projections that restore operator, action kind, and the public semantic surface collapse to the reference-equivalence denominator. The result is not that

Table 14: The fields that explain collision witnesses also repair them: placement resolves keyword loss, layer resolves operator-only loss, and the fixed basis covers every headline projection.

Projection	Single-field repairs	Top attribution fields
Keyword	variant, placement	variant=.168; placement=.168
Yes/No	variant, layer, placement, state	each=.250
Operator-only	variant, layer	variant=.184; layer=.184

Table 15: Projection repair ladder showing residual false pairs after restoring fields. Bold marks certified zero-collision repairs; underlining marks unrepaired unsafe baseline rows.

Projection	Restored fields	Eq.	False
Keyword	–	<u>2,284</u>	<u>254</u>
Keyword	operator	2,036	6
Keyword	layer	2,072	42
Keyword	placement	2,030	0
Keyword	layer+placement	2,030	0
Yes/No	–	2,036	6
Yes/No	layer	2,030	0
Yes/No	placement	2,030	0
Operator	–	<u>2,268</u>	<u>238</u>
Operator	kind	2,036	6
Operator	placement	2,062	32
Operator	layer	2,030	0
Operator	layer+placement	2,030	0

every abstraction is bad; it is that common optimizer-comparison abstractions omit query-processing fields that several independently weakened projections need and several strengthened projections restore.

Repair certificates identify the missing semantics. The projection audit makes repair checkable, but the empirical question is which fields are sufficient. The repair column in Table 7 reports minimum-cardinality universal repairs and held-out-probe generalization. Placement repairs every keyword collision. Layer repairs every operator-only collision. The learned repairs resolve all held-out collisions across probe folds. Within the declared field universe and admitted public-contract corpus, a fixed semantic basis consisting of optimizer layer and execution placement repairs all observed collisions across keyword, yes/no, and operator-only projections, even when fine-grained action-variant labels are excluded. The certificate is checked as a finite proof obligation: each counted witness is reference-signature unequal, projection-equal, separated by the reported repair, and protected by a grounded counter-witness against every smaller universal repair. The hitting-set view over all 498 headline witnesses agrees exactly with direct repaired-signature enumeration.

Table 16 adds a stronger anti-overfitting test. Each fold withholds a non-default query-feature family, learns repairs from the remaining collision witnesses, and tests them on the held-out family. A point minimum can choose an underspecified field when the training denominator is narrow; the fixed layer and placement basis resolves every held-out collision across all folds. The repair is

Table 16: Held-out feature-family repair stability for point minima and the fixed basis. Bold marks zero held-out residuals; underlining marks unrepaired unsafe residual evidence under underspecified point minima.

Projection	Folds	Held-out	Point	Basis
Keyword	12	830	<u>144</u>	0
Operator	9	569	<u>12</u>	0
Yes/No	1	3	0	0

Repair frontier over retained fields

scope	false	depth	0	1	2+	minimum basis	safe/unsafe
keyword	254	U	S	–	–	placement	23/9
yes/no	6	U	S	–	–	layer or placement	24/8
operator-only	238	U	S	–	–	layer	22/10
all	498	U	U	S	–	layer + placement	21/11
		U unsafe	S first safe	– not first			

Figure 6: Semantic frontier over retained fields. Rose U cells leave unsafe counter-witnesses; teal S cells mark first safe retained-field sets; gray dashes are deeper retained-field sets that are not the first safe basis.

therefore not a post-hoc fit to one query slice, but a stable contract denominator.

The repair ladder in Table 15 gives a stepwise view of the same diagnosis. Restoring operator alone nearly repairs keyword projection but leaves six collisions; placement eliminates them. For operator-only projection, layer eliminates all collisions while placement alone leaves residual errors. Figure 6 then reports the finite field-lattice frontier: safe subsets repair every collision in scope, unsafe subsets retain grounded counter-witnesses, and the all-projection row requires layer+placement for all 498 headline witnesses.

Table 14 shows why the repair fields are query-processing fields. Placement separates local execution, source delegation, distributed workers, and cloud-service execution. Layer and decision time separate logical rewrite, physical planning, distributed planning, and runtime adaptation. Operator and observability keep plan operators, plan evidence, physical estimates, and runtime profiles from being treated as interchangeable. The highest-attribution fields are the same fields that repair the collisions, and a fixed layer+placement semantic basis resolves every observed engine-pair hotspot without relying on fine-grained variant identifiers.

Collisions have mechanisms and hotspots. Table 17 groups collision witnesses into optimizer mechanisms and lists the largest engine-pair hotspots. The dominant mechanisms are action-variant, execution-placement, optimizer-layer, plan-evidence, and decision-time collisions. The hotspots are not random; they occur where public contracts expose different query-processing surfaces, such as source delegation versus local planning, runtime adaptivity versus compile-time planning, and stage-graph observability versus estimated local plans.

Table 17: Collision witnesses concentrate in interpretable optimizer mechanisms rather than random engine pairs; action variant, placement, and layer differences dominate the hotspots.

Mechanism	W.	Hotspot	False
Action variant	498	Keyword BQ-CH	203
Placement	466	Keyword BQ-Trino	45
Optimizer layer	430	Keyword CH-Trino	6
Plan evidence	366	Yes/No CH-Trino	6
Decision time	362	Operator CH-Trino	123
Operator family	281	Operator Spark-Trino	115

Table 18: Workload-feature contract movements under controlled toggles. Bold marks the largest mean movement; underlining marks the broadest nonzero movement.

Engine	Feature	Mean dist.	Nonzero
DuckDB	reuse_structure	0.827	0.854
Trino	statistics_need	0.742	0.853
DuckDB	source_boundary	0.590	0.623
Trino	distribution_trigger	0.519	0.861
Spark SQL	adaptivity_trigger	0.442	0.858
BigQuery	distribution_trigger	0.373	<u>0.891</u>
Spark SQL	join_type	0.369	0.810
ClickHouse	order_limit	0.351	0.511

Optimizer contracts are workload-sensitive. Table 18 reports controlled feature-toggle movements over CTE reuse, source boundary, statistics need, distribution triggers, and adaptivity triggers. Comparability is therefore a relation between engine contracts and query structure, not an engine-level scalar.

Public optimizer sources populate the nontrivial kernel; the information profile, frontier certificate, and SQL validation show a distinguishable, repairable, executable denominator.

5.1 Concrete Collision Audits

Table 19 closes the evaluation with four concrete audits from the witness ledger behind Tables 7 and 17. C1 is the running P0009 example: a coarse pushdown+explain label hides BigQuery filter delegation [16], BigQuery stage evidence [17], Trino connector pushdown [34], and Trino dynamic filtering [33]. C2-C4 cover estimate observability, runtime adaptation, and CTE materialization; the same finite checker covers all 498 headline witnesses.

6 DISCUSSION AND LIMITATIONS

Optimizer interfaces as semantic interfaces. Public optimizer claims should name action, guard, layer, placement, decision time, and evidence surface. These fields do not expose private cost formulas; they keep adapters, remote sources, and service stages from collapsing into one phenomenon, a boundary already visible in federation [8] and polystores [25].

What changes in an evaluation. The concrete change is the denominator of an optimizer study. Before ranking latency, cardinality error, or plan quality, a study can test whether compared systems share the same public contract relation. If a pushdown

Table 19: Concrete collision audits linking coarse labels to hidden contract pairs. C1 binds BigQuery federated pushdown [16] and plan-stage evidence [17] to Trino pushdown [34] and dynamic-filtering evidence [33].

Case	Hidden contract contrast	Separating fields
C1	pushdown/explain: BigQuery delegation+stage evidence vs. Trino pushdown+dynamic filtering	variant, placement, decision time
C2	plan evidence: PostgreSQL estimates vs. BigQuery/Snowflake stages	observability, placement
C3	adaptive: compile-time planning vs. runtime repartition/AQE/conversion	layer, decision time
C4	CTE: DuckDB/PostgreSQL inline, materialize, reuse, and disable guards	variant, guard state

table equates connector delegation with runtime filtering, the study can retain the separating fields, narrow the claim, or report mixed optimizer phenomena.

Protocol, scope, and refresh. Every optimizer comparison is a projection: choose a vocabulary, extract public contracts, validate the denominator, compute the kernel, and publish retained fields or assumptions. The evidence unit is a public contract, not inferred private state. Without manuals, plans, diagnostics, connector behavior, or observable SQL behavior, a rule is outside scope; counts are lower bounds and later evidence becomes a source refresh.

Threats and limits. Source selection can miss behavior vendors do not expose, so OptSem-C reports lower bounds over public contracts. Human coding can misread a span; source identifiers, line ranges, digest prefixes, side-balanced support, and regenerated table-source checks mitigate that risk. SQL execution validates visible plans, not private costs; closed systems need a separate admission policy.

7 RELATED WORK

Optimizer architecture and extensible planning. OptSem-C’s fields follow optimizer systems: System R exposes access paths and costing [65]; Volcano [36] and Cascades [37] expose rules, layers, and search; and classical surveys frame costing and architecture [10, 48]. Calcite [27], DataFusion [29], Trino pushdown [34], and BigQuery plans [17] show why placement and observability remain visible.

Equivalence, testing, and evidence. SQL-equivalence and containment work decide result preservation [1, 9, 14, 15, 72]; DBMS testers seek wrong answers or crashes [63, 64, 66, 68]. Provenance separates support from behavior [7, 39, 40], while JOB [53], TPC-H [21], and CardBench [13] define workload or estimation denominators; OptSem-C tests the public optimizer-contract denominator those studies assume.

8 CONCLUSION

OptSem-C makes optimizer comparisons replayable by turning each coarse-label collision into a public-evidence certificate with hidden distinctions and repair fields.

REFERENCES

- [1] A. V. Aho, Y. Sagiv, and J. D. Ullman. 1979. Equivalences among Relational Expressions. *SIAM J. Comput.* 8, 2 (1979), 218–246. <https://doi.org/10.1137/0208017>
- [2] Michael Armbrust, Reynold S. Xin, Cheng Lian, Yin Huai, Davies Liu, Joseph K. Bradley, Xiangrui Meng, Tomer Kaftan, Michael J. Franklin, Ali Ghodsi, and Matei Zaharia. 2015. Spark SQL: Relational Data Processing in Spark. In *SIGMOD*. 1383–1394. <https://doi.org/10.1145/2723372.2742797>
- [3] R. Avnur and J. M. Hellerstein. 2000. Eddies: Continuously Adaptive Query Processing. In *SIGMOD*. 261–272. <https://doi.org/10.1145/342009.335420>
- [4] E. Begoli, J. Camacho-Rodriguez, J. Hyde, M. J. Mior, and D. Lemire. 2018. Apache Calcite: A Foundational Framework for Optimized Query Processing Over Heterogeneous Data Sources. In *SIGMOD*. 221–230. <https://doi.org/10.1145/3183713.3190662>
- [5] D. Bitton, D. J. DeWitt, and C. Turbyfill. 1983. Benchmarking Database Systems: A Systematic Approach. In *Vldb*. 8–19. <https://www.vldb.org/conf/1983/P008.PDF>
- [6] P. A. Boncz, M. Zukowski, and N. Nes. 2005. MonetDB/X100: Hyper-Pipelining Query Execution. In *CIDR*. 225–237. <https://www.cidrdb.org/cidr2005/papers/P19.pdf>
- [7] P. Buneman, S. Khanna, and W.-C. Tan. 2001. Why and Where: A Characterization of Data Provenance. In *ICDT*. 316–330. https://doi.org/10.1007/3-540-44503-X_20
- [8] Michael J. Carey, Laura M. Haas, Peter M. Schwarz, Manish Arya, William F. Cody, Ronald Fagin, Myron Flickner, Allen W. Luniewski, Wayne Niblack, Dragutin Petkovic, John Thomas, John H. Williams, and Edward L. Wimmers. 1995. Towards Heterogeneous Multimedia Information Systems: The Garlic Approach. In *RIDE-DOM*. 124–131. <https://doi.org/10.1109/RIDE.1995.378736>
- [9] A. K. Chandra and P. M. Merlin. 1977. Optimal Implementation of Conjunctive Queries in Relational Data Bases. In *STOC*. 77–90. <https://doi.org/10.1145/800105.803397>
- [10] S. Chaudhuri. 1998. An Overview of Query Optimization in Relational Systems. In *PODS*. 34–43. <https://doi.org/10.1145/275487.275492>
- [11] S. Chaudhuri, V. R. Narasayya, and R. Ramamurthy. 2009. Exact Cardinality Query Optimization for Optimizer Testing. *PVLDB* 2, 1 (2009), 994–1005. <https://doi.org/10.14778/1687627.1687739>
- [12] J. Cheney, L. Chiticariu, and W.-C. Tan. 2009. Provenance in Databases: Why, How, and Where. *Foundations and Trends in Databases* 1, 4 (2009), 379–474. <https://doi.org/10.1561/1900000006>
- [13] Y. Chronis, Y. Wang, Y. Gan, S. Abu-El-Haija, C. Lin, C. Binnig, and F. Özcan. 2024. CardBench: A Benchmark for Learned Cardinality Estimation in Relational Databases. arXiv:2408.16170 <https://arxiv.org/abs/2408.16170> Accessed 2026-06-16.
- [14] S. Chu, B. Murphy, J. Roesch, A. Cheung, and D. Suciu. 2018. Axiomatic Foundations and Algorithms for Deciding Semantic Equivalences of SQL Queries. *PVLDB* 11, 11 (2018), 1482–1495. <https://doi.org/10.14778/3236187.3236200>
- [15] S. Chu, C. Wang, K. Weitz, and A. Cheung. 2017. Cosette: An Automated Prover for SQL. In *CIDR*. 1–10. <https://www.cidrdb.org/cidr2017/papers/p51-chu-cidr17.pdf>
- [16] Google Cloud. 2026. Introduction to Federated Queries. Documentation. <https://docs.cloud.google.com/bigquery/docs/federated-queries-intro> Accessed 2026-07-04.
- [17] Google Cloud. 2026. Query Plan and Timeline. Documentation. <https://cloud.google.com/bigquery/docs/query-plan-explanation> Accessed 2026-06-16.
- [18] E. F. Codd. 1970. A Relational Model of Data for Large Shared Data Banks. *Commun. ACM* 13, 6 (1970), 377–387. <https://doi.org/10.1145/362384.362685>
- [19] R. L. Cole and G. Graefe. 1994. Optimization of Dynamic Query Evaluation Plans. In *SIGMOD*. 150–160. <https://doi.org/10.1145/191839.191872>
- [20] Transaction Processing Performance Council. 2024. TPC Benchmark DS Standard Specification. TPC. <https://www.tpc.org/tpcds/> Accessed 2026-06-16.
- [21] Transaction Processing Performance Council. 2024. TPC Benchmark H Standard Specification. TPC. <https://www.tpc.org/tpch/> Accessed 2026-06-16.
- [22] Benoit Dageville, Thierry Cruanes, Marcin Zukowski, Vadim Antonov, Artin Avanes, Jon Bock, Jonathan Claybaugh, Daniel Engovatov, Martin Hentschel, Jiansheng Huang, Allison W. Lee, Ashish Motivala, Abdul Q. Munir, Steven Pelley, Peter Povinec, Greg Rahn, Spyridon Triantafyllis, and Philipp Unterbrunner. 2016. The Snowflake Elastic Data Warehouse. In *SIGMOD*. 215–226. <https://doi.org/10.1145/2882903.2903741>
- [23] Yuhao Deng, Chengliang Chai, Lei Cao, Qin Yuan, Siyuan Chen, Yanrui Yu, Zhaoze Sun, Junyi Wang, Jiajun Li, Ziqi Cao, Kaisen Jin, Chi Zhang, Yuqing Jiang, Yuanfang Zhang, Yuping Wang, Ye Yuan, Guoren Wang, and Nan Tang. 2024. LakeBench: A Benchmark for Discovering Joinable and Unionable Tables in Data Lakes. *PVLDB* 17, 8 (2024), 1925–1938. <https://doi.org/10.14778/3659437.3659448>
- [24] A. Deshpande, Z. Ives, and V. Raman. 2007. Adaptive Query Processing. *Foundations and Trends in Databases* 1, 1 (2007), 1–140. <https://doi.org/10.1561/1900000001>
- [25] Jennie Duggan, Aaron J. Elmore, Michael Stonebraker, Magda Balazinska, Bill Howe, Jeremy Kepner, Sam Madden, David Maier, Tim Mattson, and Stan Zdonik. 2015. The BigDAWG Polystore System. *SIGMOD Record* 44, 2 (2015), 11–16. <https://doi.org/10.1145/2814710.2814713>
- [26] Orri Erling, Alex Averbuch, Josep Larriba-Pey, Hassan Chafi, Andrey Gubichev, Arnaud Prat, Minh-Duc Pham, and Peter Boncz. 2015. The LDBC Social Network Benchmark: Interactive Workload. In *SIGMOD*. 619–630. <https://doi.org/10.1145/2723372.2742786>
- [27] Apache Software Foundation. 2026. Apache Calcite Documentation. Documentation. <https://calcite.apache.org/docs/> Accessed 2026-06-16.
- [28] Apache Software Foundation. 2026. Performance Tuning: Spark 4.1.2 Documentation. Documentation. <https://spark.apache.org/docs/4.1.2/sql-performance-tuning.html> Accessed 2026-06-16.
- [29] Apache Software Foundation. 2026. Reading Explain Plans. Documentation. <https://datafusion.apache.org/user-guide/explain-usage.html> Accessed 2026-06-16.
- [30] DuckDB Foundation. 2026. DuckDB Documentation: WITH Clause and CTE Materialization. Documentation. https://duckdb.org/docs/stable/sql/query_syntax/with.html Accessed 2026-06-16.
- [31] Presto Foundation. 2026. Cost Based Optimizations: Presto 0.298.1 Documentation. Documentation. <https://prestodb.github.io/docs/current/optimizer/cost-based-optimizations.html> Accessed 2026-06-16.
- [32] Trino Software Foundation. 2026. Cost-Based Optimizations: Trino 482 Documentation. Documentation. <https://trino.io/docs/482/optimizer/cost-based-optimizations.html> Accessed 2026-06-16.
- [33] Trino Software Foundation. 2026. Dynamic Filtering: Trino 482 Documentation. Documentation. <https://trino.io/docs/current/admin/dynamic-filtering.html> Accessed 2026-07-04.
- [34] Trino Software Foundation. 2026. Pushdown: Trino 482 Documentation. Documentation. <https://trino.io/docs/current/optimizer/pushdown.html> Accessed 2026-07-04.
- [35] Cesar A. Galindo-Legaria and Arnon Rosenthal. 1997. Outerjoin Simplification and Reordering for Query Optimization. *ACM Transactions on Database Systems* 22, 1 (1997), 43–74. <https://doi.org/10.1145/244810.244812>
- [36] G. Graefe. 1990. Encapsulation of Parallelism in the Volcano Query Processing System. In *SIGMOD*. 102–111. <https://doi.org/10.1145/93597.98720>
- [37] G. Graefe. 1995. The Cascades Framework for Query Optimization. *IEEE Data Engineering Bulletin* 18, 3 (1995), 19–29. <https://www.sigmod.org/publications/dblp/db/journals/debu/Graefe95a.html>
- [38] G. Graefe and W. J. McKenna. 1993. The Volcano Optimizer Generator: Extensibility and Efficient Search. In *ICDE*. 209–218. <https://doi.org/10.1109/ICDE.1993.344061>
- [39] T. J. Green. 2011. Containment of Conjunctive Queries on Annotated Relations. *Theory of Computing Systems* 49, 2 (2011), 429–459. <https://doi.org/10.1007/s00224-011-9327-6>
- [40] T. J. Green, G. Karvounarakis, and V. Tannen. 2007. Provenance Semirings. In *PODS*. 31–40. <https://doi.org/10.1145/1265530.1265535>
- [41] PostgreSQL Global Development Group. 2026. PostgreSQL Documentation: 18: 14.1. Using EXPLAIN. Documentation. <https://www.postgresql.org/docs/18/using-explain.html> Accessed 2026-06-16.
- [42] Anurag Gupta, Deepak Agarwal, Derek Tan, Jakub Kulesza, Rahul Pathak, Stefano Stefani, and Vidhya Srinivasan. 2015. Amazon Redshift and the Case for Simpler Data Warehouses. In *SIGMOD*. 1917–1923. <https://doi.org/10.1145/2723372.2742795>
- [43] Yuxing Han, Ziniu Wu, Peizhi Wu, Rong Zhu, Jingyi Yang, Liang Wei Tan, Kai Zeng, Gao Cong, Yanzhao Qin, Andreas Pfadler, Zhengping Qian, Jingren Zhou, Jiangneng Li, and Bin Cui. 2021. Cardinality Estimation in DBMS: A Comprehensive Benchmark Evaluation. *PVLDB* 15, 4 (2021), 752–765. <https://doi.org/10.14778/3503585.3503586>
- [44] B. Hilprecht, A. Schmidt, M. Kulesa, A. Molina, K. Kersting, and C. Binnig. 2020. DeepDB: Learn from Data, Not from Queries! *PVLDB* 13, 7 (2020), 992–1005. <https://doi.org/10.14778/3384345.3384349>
- [45] ClickHouse Inc. 2024. ClickBench: A Benchmark for Analytical DBMS. Online benchmark. <https://benchmark.clickhouse.com/> Accessed 2026-06-16.
- [46] ClickHouse Inc. 2026. Using JOINS in ClickHouse. Documentation. <https://clickhouse.com/docs/guides/joining-tables> Accessed 2026-06-16.
- [47] Snowflake Inc. 2026. EXPLAIN: Snowflake Documentation. Documentation. <https://docs.snowflake.com/en/sql-reference/sql/explain> Accessed 2026-06-16.
- [48] Y. E. Ioannidis. 1996. Query Optimization. *Comput. Surveys* 28, 1 (1996), 121–123. <https://doi.org/10.1145/234313.234367>
- [49] Z. G. Ives, D. Florescu, M. Friedman, A. Levy, and D. S. Weld. 1999. An Adaptive Query Execution System for Data Integration. *ACM SIGMOD Record* 28, 2 (1999), 299–310. <https://doi.org/10.1145/304181.304209>
- [50] David Justen, Daniel Ritter, Campbell Fraser, Andrew Lamb, Allison Lee, Thomas Bodner, Mhd Yamen Haddad, Steffen Zeuch, Volker Markl, and Matthias Boehm. 2024. POLAR: Adaptive and Non-invasive Join Order Selection via Plans of Least Resistance. *PVLDB* 17, 6 (2024), 1350–1363. <https://doi.org/10.14778/3648160.3648175>

- [51] N. Kabra and D. J. DeWitt. 1998. Efficient Mid-Query Re-Optimization of Sub-Optimal Query Execution Plans. In *SIGMOD*. 106–117. <https://doi.org/10.1145/276304.276315>
- [52] Andreas Kipf, Thomas Kipf, Bernhard Radke, Viktor Leis, Peter Boncz, and Alfons Kemper. 2019. Learned Cardinalities: Estimating Correlated Joins with Deep Learning. In *CIDR*. 1–11. <https://www.cidrdb.org/cidr2019/papers/p101-kipf-cidr19.pdf>
- [53] V. Leis, A. Gubichev, A. Mirchev, P. Boncz, A. Kemper, and T. Neumann. 2015. How Good Are Query Optimizers, Really? *PVLDB* 9, 3 (2015), 204–215. <https://doi.org/10.14778/2850583.2850594>
- [54] V. Leis, B. Radke, A. Gubichev, A. Mirchev, P. Boncz, A. Kemper, and T. Neumann. 2018. Query Optimization Through the Looking Glass, and What We Found Running the Join Order Benchmark. *Vldb Journal* 27, 5 (2018), 643–668. <https://doi.org/10.1007/s00778-017-0480-7>
- [55] Ryan Marcus, Parimarjan Negi, Hongzi Mao, Nesime Tatbul, Mohammad Alizadeh, and Tim Kraska. 2021. Bao: Making Learned Query Optimization Practical. In *SIGMOD*. 1275–1288. <https://doi.org/10.1145/3448016.3452838>
- [56] R. Marcus and O. Papaemmanouil. 2019. Neo: A Learned Query Optimizer. *PVLDB* 12, 11 (2019), 1705–1718. <https://doi.org/10.14778/3342263.3342644>
- [57] V. Markl, V. Raman, D. Simmen, G. Lohman, H. Pirahesh, and M. Cilimdžić. 2004. Robust Query Processing Through Progressive Optimization. In *SIGMOD*. 659–670. <https://doi.org/10.1145/1007568.1007642>
- [58] Sergey Melnik, Andrey Gubarev, Jing Jing Long, Geoffrey Romer, Shiva Shivakumar, Matt Tolton, and Theo Vassilakis. 2010. Dremel: Interactive Analysis of Web-Scale Datasets. *PVLDB* 3, 1-2 (2010), 330–339. <https://doi.org/10.14778/1920841.1920886>
- [59] G. Moerkotte and T. Neumann. 2006. Analysis of Two Existing and One New Dynamic Programming Algorithm for the Generation of Optimal Bushy Join Trees without Cross Products. In *Vldb*. 930–941. <https://dblp.org/rec/conf/vldb/MoerkotteN06.html>
- [60] T. Neumann. 2011. Efficiently Compiling Efficient Query Plans for Modern Hardware. *PVLDB* 4, 9 (2011), 539–550. <https://doi.org/10.14778/2002938.2002940>
- [61] P. O’Neil, E. O’Neil, X. Chen, and S. Revilak. 2009. The Star Schema Benchmark and Augmented Fact Table Indexing. In *TPCTC*. 237–252. https://doi.org/10.1007/978-3-642-10424-4_17
- [62] M. Raasveldt and H. Muehleisen. 2019. DuckDB: An Embeddable Analytical Database. In *SIGMOD*. 1981–1984. <https://doi.org/10.1145/3299869.3320212>
- [63] M. Rigger and Z. Su. 2020. Detecting Optimization Bugs in Database Engines via Non-Optimizing Reference Engine Construction. In *ESEC/FSE*. 1140–1152. <https://doi.org/10.1145/3368089.3409710>
- [64] Manuel Rigger and Zhendong Su. 2020. Testing Database Engines via Pivoted Query Synthesis. In *OSDI*. USENIX Association, 667–682. <https://www.usenix.org/conference/osdi20/presentation/rigger>
- [65] P. G. Selinger, M. M. Astrahan, D. D. Chamberlin, R. A. Lorie, and T. G. Price. 1979. Access Path Selection in a Relational Database Management System. In *SIGMOD*. 23–34. <https://doi.org/10.1145/582095.582099>
- [66] Andreas Seltenreich. 2015. SQLsmith: Randomized SQL Testing. <https://github.com/anse1/sqlsmith> Accessed 2026-07-04.
- [67] Jeff Shute, Radek Vingralek, Bart Samwel, Ben Handy, Chad Whipkey, Eric Rollins, Mircea Oancea, Kyle Littlefield, David Menestrina, Stephan Ellner, John Cieslewicz, Ian Rae, Traian Stancescu, and Himani Apte. 2013. F1: A Distributed SQL Database That Scales. *PVLDB* 6, 11 (2013), 1068–1079. <https://doi.org/10.14778/2536222.2536232>
- [68] Donald R. Slutz. 1998. Massive Stochastic Testing of SQL. In *Vldb*. 618–622. <https://www.vldb.org/conf/1998/p618.pdf>
- [69] Mohamed A. Soliman, Lyublena Antova, Venkatesh Raghavan, Amr El-Helw, Zhongxian Gu, Entong Shen, George C. Caragea, Carlos Garcia-Alvarado, Foyzur Rahman, Michalis Petropoulos, Florian Waas, Sivaramakrishnan Narayanan, Konstantinos Krikellias, and Rhonda Baldwin. 2014. Orca: A Modular Query Optimizer Architecture for Big Data. In *SIGMOD*. 337–348. <https://doi.org/10.1145/2588555.2595637>
- [70] M. Steinbrunn, G. Moerkotte, and A. Kemper. 1997. Heuristic and Randomized Optimization for the Join Ordering Problem. *Vldb Journal* 6, 3 (1997), 191–208. <https://doi.org/10.1007/s007780050040>
- [71] C. Turbyfill, C. Orji, and D. Bitton. 1993. AS3AP: An ANSI SQL Standard Scaleable and Portable Benchmark for Relational Database Systems. In *The Benchmark Handbook for Database and Transaction Processing Systems*. 95–121. <https://www.odbms.org/2014/03/benchmark-handbook-1993/>
- [72] S. Wang, S. Pan, and A. Cheung. 2024. QED: A Powerful Query Equivalence Decider for SQL. *PVLDB* 17, 11 (2024), 3602–3614. <https://doi.org/10.14778/3681954.3682024>
- [73] Z. Wu, P. Negi, M. Alizadeh, T. Kraska, and S. Madden. 2023. FactorJoin: A New Cardinality Estimation Framework for Join Queries. *Proceedings of the ACM on Management of Data* 1, 1 (2023), 1–27. <https://doi.org/10.1145/3588721>
- [74] Z. Yang, E. Liang, A. Kamsetty, C. Wu, Y. Duan, X. Chen, P. Abbeel, J. M. Hellerstein, S. Krishnan, and I. Stoica. 2019. Deep Unsupervised Cardinality Estimation. *PVLDB* 13, 3 (2019), 279–292. <https://doi.org/10.14778/3368289.3368294>
- [75] Matei Zaharia, Mosharaf Chowdhury, Tathagata Das, Ankur Dave, Justin Ma, Murphy McCauley, Michael J. Franklin, Scott Shenker, and Ion Stoica. 2012. Resilient Distributed Datasets: A Fault-Tolerant Abstraction for In-Memory Cluster Computing. In *NSDI*. 15–28. <https://www.usenix.org/conference/nsdi12/technical-sessions/presentation/zaharia>